Homework

A homework for class **MET CS 763** at Boston University, instructed by **Yuting Zhang**. Student Zhang, Jin Yu submitted on **April 11, 2023**.

Problem 1

Answer the questions based on the following program:

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

#define MAX_LEN 256

int main(int argc, char* argv[]) {
    char *greetingMsg = malloc(MAX_LEN);
    char name[MAX_LEN];

    char greeting[] = "Hello!";
    int len = 0;

    greetingMsg[0] = '\0';

    strncat(greetingMsg, greeting, sizeof(greeting));

    int i = 1;
    while (i < argc) {
        strncpy(name, argv[i], strlen(argv[i]));
        if (i > 1)
            strncat(greetingMsg, ", ", 1);
        strncat(greetingMsg, name, strlen(name));
        i++;
    }
    strncat(greetingMsg, "\n", 1);
    printf(greetingMsg);
    return 0;
}
```

What are variables in this program? Where is each variable stored:

- `greetingMsg`: Heap. A pointer to a dynamically allocated memory block with a size of *MAX_LEN*

- `name`: Stack. A character array with a size of *MAX_LEN*, which is used to temporarily store the names from the command-line arguments
- `greeting`: Stack. Array of characters that contains the initial greeting message *"Hello!"*
- `len`: Stack. Integer variable that is not currently used in the program
- `i`: Stack. Integer variable used as a counter in the while loop
- `argc`: Stack. Integer variable holds arguments
- `argv`: Stack. A pointer to an array of character pointers

What are bugs/vulnerabilities in this program? Briefly describe each of them:

- **Missing *free()* for greetingMsg**: The program allocates memory for the greetingMsg pointer using *malloc()*, but it never frees the memory with *free()*. This leads to a memory leak, as the allocated memory is not released after the program execution
- ***strncat()* Misuse for ", "**: The *strncat()* function is called with a size argument of 1 to append a ", " string to greetingMsg. This will cause the null-terminator to be overwritten, potentially leading to a buffer overflow when concatenating further strings. The correct size argument should be **2**
- ***printf()* with a Non-Constant Format String**: The *printf()* function is called with greetingMsg as its format string. This can lead to format string vulnerabilities if the input contains any format specifiers (e.g., %s, %d)
- **Buffer Overflow with *strncat()***: The program uses *strncat()* to concatenate strings, but it does not ensure that the destination buffer (greetingMsg) has enough space for the concatenated strings. If the combined length of the command-line arguments exceeds *MAX_LEN*, a buffer overflow may occur
- **Improper Use of *strncpy()***: The *strncpy()* function is used to copy the name from argv[i] to the name buffer. However, *strncpy()* does not guarantee a null-terminated string if the source string's length is equal to or greater than the specified size. This can lead to undefined behavior when working with the name buffer later

Problem 2

Suppose Alice wants to communicate with Bob, please describe all necessary crypto means to provide confidentiality and integrity for this communication channel. In particular, your solution should prevent MITM. Please also state your assumptions:

1. **Public Key Infrastructure (PKI)**: Assume that both Alice and Bob have public/private key pairs, and their public keys are signed by a trusted Certificate Authority (CA). This helps to establish trust in the public keys, preventing MITM attacks where an attacker poses as Alice or Bob
2. **Secure Key Exchange**: Alice and Bob use a secure key exchange protocol, such as the Diffie-Hellman Key Exchange or Elliptic Curve Diffie-Hellman (ECDH), to establish a shared secret key. This shared secret will be used for symmetric encryption to provide confidentiality. The key exchange protocol ensures that even if an attacker intercepts the exchanged messages, they cannot derive the shared secret key
3. **Symmetric Encryption**: Alice and Bob use the shared secret key to encrypt and decrypt messages using a symmetric encryption algorithm like AES (Advanced Encryption

Standard). This provides confidentiality for the communication channel, ensuring that only Alice and Bob can read each other's messages

4. **Digital Signatures**: To provide integrity and authentication, Alice and Bob use their private keys to sign their messages using a digital signature algorithm, such as RSA or ECDSA (Elliptic Curve Digital Signature Algorithm). The recipient (Alice or Bob) verifies the digital signature using the sender's public key, ensuring that the message has not been tampered with and was sent by the intended party

Assumptions:

1. Alice and Bob have secure public/private key pairs, and their public keys have been signed by a trusted CA
2. Alice and Bob have access to secure implementations of cryptographic algorithms and protocols
3. Alice and Bob's devices are secure, and their private keys are stored safely
4. The communication channel may be intercepted or monitored by attackers, but the cryptographic mechanisms in place protect the confidentiality and integrity of the messages

Problem 3

Please read the paper “Analysis of the HTTPS Certificate Ecosystem“ that is posted on Blackboard the assignment page and answer the following questions:

What is PKI?

Public Key Infrastructure (PKI) is a set of policies, procedures, hardware, software, and services that provide a framework for creating, managing, distributing, using, storing, and revoking digital certificates and public-key cryptography. PKI enables secure communication and authentication over the internet by establishing trust in the identity of entities (e.g., individuals, servers, devices, etc.) through a trusted third party called a Certificate Authority (CA).

What are the differences and relationships between digital signature and digital certificate?

A digital signature is a cryptographic technique that uses public key cryptography to sign and verify digital data. It ensures the integrity, authenticity, and non-repudiation of the data. The sender signs the data using their private key, and the recipient verifies the signature using the sender's public key. Digital signatures help ensure that the data has not been tampered with and that it originates from the stated sender.

A digital certificate (also known as a public key certificate) is an electronic document issued by a CA that binds the identity of an entity (e.g., a person, organization, or server) to a public key. The certificate contains information about the entity, the public key, the CA that issued the certificate, and a digital signature from the CA. Digital certificates enable secure communication and authentication by allowing the exchange of public keys and establishing trust in the identity of the communicating parties.

The relationship between digital signatures and digital certificates lies in the establishment of trust. A digital certificate is used to share a public key and verify the identity of the entity associated with that public key. The digital signature of the CA on the certificate ensures the authenticity and integrity of the certificate, and by extension, the public key contained within it.

What is HTTPS? How is digital signature/digital certificate used?

HTTPS (Hypertext Transfer Protocol Secure) is an extension of HTTP (Hypertext Transfer Protocol) that provides secure communication over the internet. It uses Transport Layer Security (TLS) or Secure Sockets Layer (SSL) protocols to encrypt and authenticate the communication between the client (e.g., web browser) and the server.

Digital signatures and digital certificates are used in HTTPS to establish trust and secure communication. When a client connects to an HTTPS server, the server presents its digital certificate, which contains the server's public key and is signed by a trusted CA. The client verifies the digital signature on the certificate, ensuring the certificate's authenticity and integrity. This process establishes trust in the server's identity and allows the client to securely exchange a symmetric encryption key with the server for encrypting the communication.

Write one/two paragraphs to state issues in the HTTPS ecosystem described in the paper.

The paper "Analysis of the HTTPS Certificate Ecosystem" highlights several issues in the HTTPS ecosystem, including the complexity and inconsistencies in the CA landscape, misissued certificates, and the susceptibility of HTTPS to attacks. The authors observed that the certificate ecosystem was highly concentrated, with a small number of CAs responsible for a significant percentage of issued certificates. This centralization raises concerns about trust and the potential for single points of failure. Additionally, the authors found instances of misissued certificates, which could undermine trust in the system, and highlighted vulnerabilities in the HTTPS ecosystem that could be exploited by attackers, such as Man-in-the-Middle (MITM) attacks.

This paper was published in 2013. Do some research on the internet to find any solution that has been developed or used to address the issues and improve the situation in recent couple of years.

- **Let's Encrypt**: Launched in 2016, Let's Encrypt is a free, automated, and open CA that aims to make obtaining and managing SSL/TLS certificates easier and more accessible. By simplifying the process, Let's Encrypt has contributed to a significant increase in the number of websites using HTTPS
- **Certificate Transparency (CT)**: Certificate Transparency is an open framework developed by Google to monitor and audit SSL/TLS certificates in real-time. It uses public, append-only logs to record issued certificates, allowing anyone to check the logs for misissued or malicious certificates. CT helps detect fraudulent certificates and improve the overall security of the HTTPS ecosystem. Since April 2018, Google Chrome requires all new SSL/TLS certificates to be CT-qualified
- **DNS-based Authentication of Named Entities (DANE)**: DANE is a protocol that uses the DNS Security Extensions (DNSSEC) to enable domain owners to specify which CA can

issue certificates for their domain. This reduces the risk of rogue CAs issuing fraudulent certificates and strengthens the trust in the certificate ecosystem. DANE allows for more secure and controlled certificate issuance and validation

- **HTTP Public Key Pinning (HPKP)**: HPKP is a security feature that allows websites to specify a set of public keys in the form of hashes that should be used for future connections. By pinning public keys, HPKP helps prevent MITM attacks using rogue certificates. However, HPKP has been deprecated due to the risk of misconfiguration causing site outages and the increased adoption of Certificate Transparency
- **OCSP Stapling**: Online Certificate Status Protocol (OCSP) Stapling is a method to check the revocation status of certificates. It reduces the overhead of OCSP requests by allowing the server to provide a signed and up-to-date OCSP response during the TLS handshake. This approach not only improves performance but also enhances privacy, as the client does not need to contact the CA directly
- **Short-Lived Certificates**: Another approach to improve the HTTPS ecosystem is the use of short-lived certificates with a validity period of only a few days. Short-lived certificates minimize the window of opportunity for attackers to exploit compromised certificates and reduce the reliance on certificate revocation mechanisms like OCSP and CRLs

References

- **Public Key Infrastructure (PKI)**: https://en.wikipedia.org/wiki/Public_key_infrastructure
- **Digital Signatures and Digital Certificates**: https://en.wikipedia.org/wiki/Digital_signature, https://en.wikipedia.org/wiki/Public_key_certificate
- **HTTPS and its Use of Digital Signatures and Certificates**: https://en.wikipedia.org/wiki/HTTPS, https://www.cloudflare.com/learning/ssl/how-does-ssl-work/
- **Let's Encrypt**: https://letsencrypt.org/
- **Certificate Transparency (CT)**: https://www.certificate-transparency.org/, https://developers.google.com/web/fundamentals/security/certificate-transparency
- **DNS-based Authentication of Named Entities (DANE)**: https://en.wikipedia.org/wiki/DNS-based_Authentication_of_Named_Entities https://tools.ietf.org/html/rfc6698
- **HTTP Public Key Pinning (HPKP)**: https://en.wikipedia.org/wiki/HTTP_Public_Key_Pinning, https://tools.ietf.org/html/rfc7469
- **OCSP Stapling**: https://en.wikipedia.org/wiki/OCSP_stapling, https://www.digicert.com/blog/ocsp-stapling/
- **Short-lived Certificates**: https://scotthelme.co.uk/short-lived-certificates-with-lets-encrypt-and-nginx/